Library Management System

Spring Boot · JPA · JSP · H2 · JUnit/Mockito

Submitted by: Krritin Keshan | Date: 02 May 2026

Contents

1. Project Overview
2. Entity Relationship Design
3. Architecture & Project Structure
4. Implementation Details
    a. Populate Database (Seed)
    b. Create Operation
    c. Read Operation (with INNER JOIN)
    d. Update Operation
5. Testing Strategy
6. Challenges Faced & How They Were Overcome
7. Build & Run Instructions
8. GitHub Repository

1. Project Overview

This project is a web-based **Library Management System** that lets a user create, read and update information about **Authors** and the **Books** they have written. It is built on Spring Boot 3.2 with Spring MVC, Spring Data JPA, JSP views, and an embedded H2 database. All required CRUD operations are exposed through styled JSP pages, and unit tests for the repository and service layers are provided using JUnit 5 and Mockito.

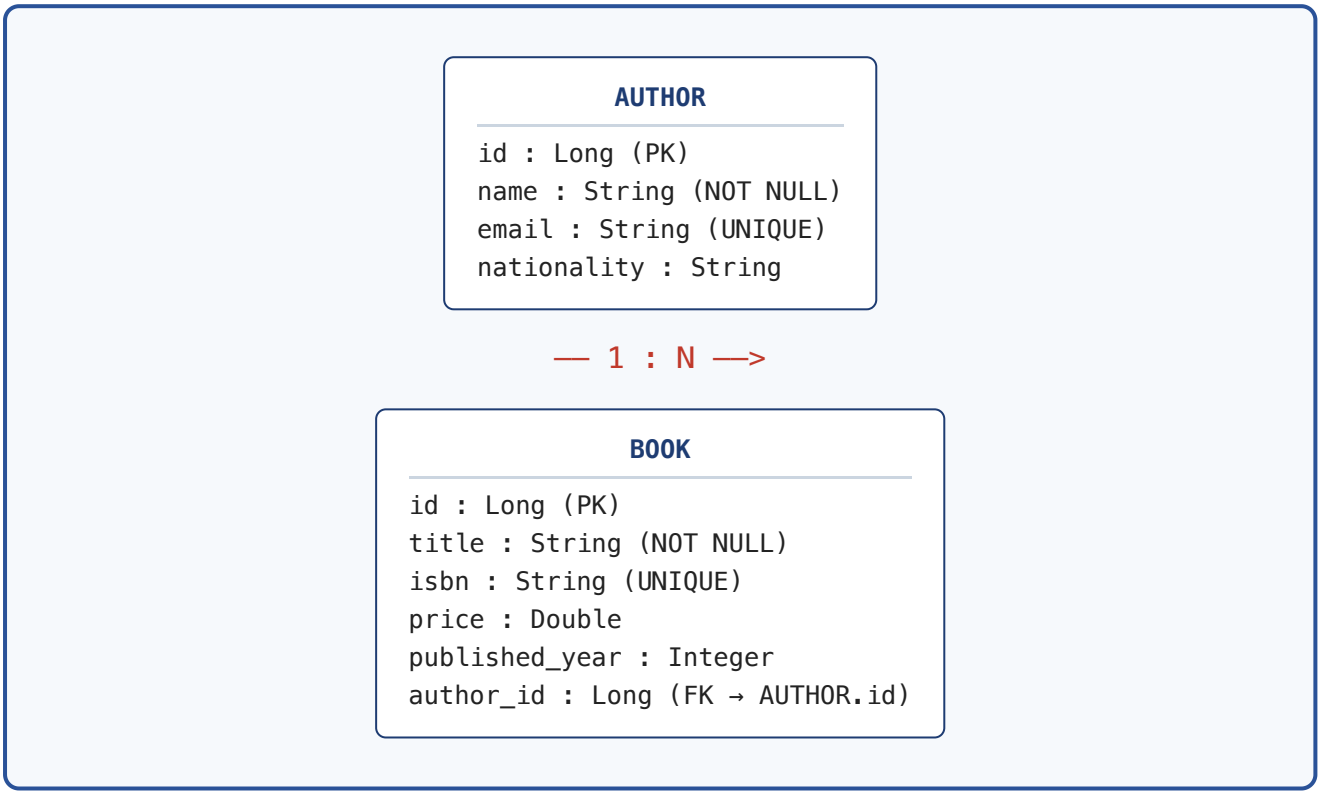
Tech Stack

Layer	Technology
Language / Runtime	Java 17, Spring Boot 3.2.5
Web	Spring MVC, JSP, JSTL, Spring `form` tags
Persistence	Spring Data JPA, Hibernate, H2 (in-memory)
Validation	Jakarta Bean Validation (`@NotBlank` , `@Email` , `@Positive`)
Build	Maven (with wrapper `./mvnw`)
Testing	JUnit 5, Mockito, Spring Boot Test, `@DataJpaTest`

Styling	Custom CSS served from `/static/css/styles.css`

2. Entity Relationship Design

Two entities were chosen: **Author** and **Book**. An author writes many books, and each book belongs to exactly one author. This is modelled as a `@OneToMany` from `Author` to `Book` and a corresponding `@ManyToOne` on the owning side (`Book`).



JPA mapping (excerpt)

```
@Entity
@Table(name = "authors")
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Column(nullable = false, length = 100)
    private String name;

    @Email @Column(unique = true, length = 150)
    private String email;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();
}

@Entity
@Table(name = "books")
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @NotBlank @Column(nullable = false, length = 200)
    private String title;

    @NotBlank @Column(unique = true, nullable = false, length = 20)
    private String isbn;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}
```

3. Architecture & Project Structure

The application follows a classical layered architecture: **Controller** → **Service** → **Repository** → **Database**. JSP views render the model returned by the controller via a `InternalResourceViewResolver` configured through Spring Boot properties.

```
src/main/java/com/bits/sga
 ├── LibraryManagementApplication.java   # @SpringBootApplication entry-point
 ├── entity/
 │     ├── Author.java                   # @Entity, @OneToMany
 │     └── Book.java                     # @Entity, @ManyToOne
 ├── dto/
 │     └── BookAuthorDTO.java            # JPQL projection for INNER JOIN result
 ├── repository/
 │     ├── AuthorRepository.java         # extends JpaRepository
 │     └── BookRepository.java           # custom @Query with INNER JOIN
 ├── service/
 │     ├── AuthorService.java
 │     └── BookService.java
 ├── controller/
 │     ├── HomeController.java
 │     ├── AuthorController.java
 │     ├── BookController.java
 │     └── GlobalExceptionHandler.java   # @ControllerAdvice
 ├── exception/ResourceNotFoundException.java
 └── config/DataInitializer.java         # CommandLineRunner that seeds 10+10 rows

src/main/resources/
 ├── application.properties              # JPA, H2, JSP view-resolver config
 └── static/css/styles.css                # styling for JSP pages

src/main/webapp/WEB-INF/views/
 ├── home.jsp
 ├── error.jsp
 ├── authors/{list,add,edit}.jsp
 └── books/{list,add,edit}.jsp

src/test/java/com/bits/sga/
 ├── repository/BookRepositoryTest.java  # @DataJpaTest, tests INNER JOIN query
 └── service/{AuthorServiceTest, BookServiceTest}.java  # Mockito unit tests
```

4. Implementation Details

4(a). Populate Database

A `CommandLineRunner` bean inside `DataInitializer` seeds **10 authors** and **10 books** the first time the application starts (it skips seeding if the table is already populated).

```
@Configuration
public class DataInitializer {

    @Bean
    public CommandLineRunner seedData(AuthorRepository authorRepository,
                                      BookRepository bookRepository) {
        return args -> {
            if (authorRepository.count() > 0) return;

            List<Author> authors = List.of(
                new Author("J.K. Rowling",        "jkrowling@books.com", "British"),
                new Author("George R.R. Martin",  "grrmartin@books.com", "American"),
                new Author("J.R.R. Tolkien",      "jrrtolkien@books.com", "British"),
                new Author("Agatha Christie",     "agatha@books.com",    "British"),
                new Author("Stephen King",        "sking@books.com",     "American"),
                new Author("Ruskin Bond",         "ruskin@books.com",    "Indian"),
                new Author("Chetan Bhagat",       "chetan@books.com",    "Indian"),
                new Author("Haruki Murakami",     "murakami@books.com",  "Japanese"),
                new Author("Paulo Coelho",        "coelho@books.com",    "Brazilian"),
                new Author("Dan Brown",           "dbrown@books.com",    "American")
            );
            List<Author> saved = authorRepository.saveAll(authors);

            bookRepository.saveAll(List.of(
                new Book("Harry Potter and the Sorcerer's Stone",
                         "978-0439708180", 499.0, 1997, saved.get(0)),
                new Book("A Game of Thrones",
                         "978-0553103540", 799.0, 1996, saved.get(1)),
                /* ... 8 more books, one per author ... */
                new Book("The Da Vinci Code",
                         "978-0307474278", 550.0, 2003, saved.get(9))
            ));
        };
    }
}
```

4(b). Create Operation

The user clicks "+ *Add Book*" on the books list page, which navigates to a JSP form bound to a `Book` model attribute via Spring's `<form:form>` tag. Submitting the form sends a `POST /books/save` request, validated by `@Valid`.

Form (books/add.jsp)

```
<form:form method="post" action="${pageContext.request.contextPath}/books/save"
           modelAttribute="book">
    <div class="form-group">
```

```
        <label for="title">Title</label>
        <form:input path="title" id="title" cssClass="form-control"/>
        <form:errors path="title" cssClass="error"/>
    </div>
    ...
    <select name="authorId" class="form-control">
        <option value="">-- Select Author --</option>
        <c:forEach var="a" items="${authors}">
            <option value="${a.id}"><c:out value="${a.name}"/></option>
        </c:forEach>
    </select>
    <button type="submit" class="btn btn-primary">Save Book</button>
</form:form>
```

Controller

```
@PostMapping("/save")
public String saveBook(@Valid @ModelAttribute("book") Book book,
                       BindingResult result,
                       @RequestParam(required = false) Long authorId,
                       Model model,
                       RedirectAttributes redirectAttributes) {
    if (authorId == null) result.rejectValue("author", "author.required",
                                             "Please select an author.");
    if (result.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        return "books/add";
    }
    try {
        bookService.save(book, authorId);
        redirectAttributes.addFlashAttribute("successMessage",
            "Book '" + book.getTitle() + "' created successfully.");
        return "redirect:/books";
    } catch (DataIntegrityViolationException ex) {
        result.rejectValue("isbn", "duplicate.isbn", ex.getMessage());
        model.addAttribute("authors", authorService.findAll());
        return "books/add";
    }
}
```

Service-side integrity check

```
public Book save(Book book, Long authorId) {
    if (book.getId() == null && book.getIsbn() != null
            && bookRepository.existsByIsbn(book.getIsbn())) {
        throw new DataIntegrityViolationException(
            "A book with ISBN '" + book.getIsbn() + "' already exists.");
    }
    Author author = authorService.findById(authorId);
    book.setAuthor(author);
    return bookRepository.save(book);
}
```

A global `@ControllerAdvice` additionally renders a friendly error page if any unexpected `DataIntegrityViolationException` escapes the controller.

4(c). Read Operation (with INNER JOIN)

Listing books uses a **custom JPQL query** in the repository that performs an **INNER JOIN** between `Book` and `Author`, projecting the result into a `BookAuthorDTO`. This avoids lazy-loading issues in the JSP and proves the join via a single SQL statement.

Repository — custom query

```
public interface BookRepository extends JpaRepository<Book, Long> {

    @Query("SELECT new com.bits.sga.dto.BookAuthorDTO(" +
           "b.id, b.title, b.isbn, b.price, b.publishedYear, " +
           "a.id, a.name, a.nationality) " +
           "FROM Book b INNER JOIN b.author a " +
           "ORDER BY a.name ASC, b.title ASC")
    List<BookAuthorDTO> findAllBooksWithAuthors();
}
```

Hibernate translates this to:

```
select b1_0.id, b1_0.title, b1_0.isbn, b1_0.price, b1_0.published_year,
       a1_0.id, a1_0.name, a1_0.nationality
  from books b1_0
  join authors a1_0 on a1_0.id = b1_0.author_id
 order by a1_0.name, b1_0.title
```

Controller binds the result to the JSP

```
@GetMapping
public String listBooks(Model model) {
    model.addAttribute("books", bookService.findAllWithAuthors());
    return "books/list";
}
```

JSP renders the joined columns using JSTL / EL

```
<c:forEach var="b" items="${books}">
    <tr>
        <td>${b.bookId}</td>
        <td><c:out value="${b.title}"/></td>
        <td><fmt:formatNumber value="${b.price}" type="currency"
                              currencySymbol="&#8377;"/></td>
        <td><c:out value="${b.authorName}"/></td>
        <td><c:out value="${b.authorNationality}"/></td>
    </tr>
</c:forEach>
```

4(d). Update Operation

Each row in the list view has an *Edit* button that links to `/books/edit/{id}` or `/authors/edit/{id}`. The controller pre-populates the form, and submission goes to `POST /books/update/{id}`, which updates the existing record by id (rather than creating a new one).

```
@PostMapping("/update/{id}")
public String updateBook(@PathVariable Long id,
                         @Valid @ModelAttribute("book") Book book,
                         BindingResult result,
                         @RequestParam(required = false) Long authorId,
                         Model model,
                         RedirectAttributes redirectAttributes) {
    if (authorId == null) result.rejectValue("author", "author.required",
                                             "Please select an author.");
    if (result.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        return "books/edit";
    }
    bookService.update(id, book, authorId);
    redirectAttributes.addFlashAttribute("successMessage",
        "Book '" + book.getTitle() + "' updated successfully.");
    return "redirect:/books";
}
```

```
// BookService.update — keeps id stable, mutates fields in-place
public Book update(Long id, Book updated, Long authorId) {
    Book existing = findById(id);
    existing.setTitle(updated.getTitle());
    existing.setIsbn(updated.getIsbn());
    existing.setPrice(updated.getPrice());
    existing.setPublishedYear(updated.getPublishedYear());
    existing.setAuthor(authorService.findById(authorId));
    return bookRepository.save(existing);
}
```

5. Testing Strategy

Two flavours of tests are provided:

1. **Service tests with Mockito** — `AuthorServiceTest` and `BookServiceTest` mock the repository and verify business logic in isolation (find, save, update, integrity-violation paths).
2. **Repository test with `@DataJpaTest`** — `BookRepositoryTest` spins up an in-memory H2 with the JPA slice and actually executes the custom `findAllBooksWithAuthors()` JPQL query, asserting that an inner join is performed and the projected `BookAuthorDTO` values are correct.

Mockito example

```
@Test
void save_shouldThrow_whenIsbnAlreadyExists() {
    Book duplicate = new Book("Duplicate", "ISBN-DUP", 100.0, 2020, null);
    when(bookRepository.existsByIsbn("ISBN-DUP")).thenReturn(true);

    assertThatThrownBy(() -> bookService.save(duplicate, 1L))
            .isInstanceOf(DataIntegrityViolationException.class)
            .hasMessageContaining("ISBN-DUP");
```

```
    verify(bookRepository, never()).save(any());
}
```

@DataJpaTest example

```
@Test
void findAllBooksWithAuthors_shouldReturnInnerJoinResults() {
    List<BookAuthorDTO> results = bookRepository.findAllBooksWithAuthors();

    assertThat(results).hasSize(2);
    assertThat(results).extracting(BookAuthorDTO::getAuthorName)
            .containsOnly("Test Author");
}
```

Test results

```
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

6. Challenges Faced & How They Were Overcome

Challenge	Resolution
Spring Boot 3 uses the `jakarta.*` namespace, but a lot of online JSP/JSTL tutorials still reference `javax.servlet.jsp.jstl`, which silently breaks JSTL tags.	Switched both the JSTL API and implementation to the Jakarta-namespace artifacts: `jakarta.servlet.jsp.jstl-api 3.0.0` and `org.glassfish.web:jakarta.servlet.jsp.jstl 3.0.1`, and updated the taglib URI in JSPs from `http://java.sun.com/jsp/jstl/core` to `jakarta.tags.core`.
Embedded Tomcat in Spring Boot does not serve `.jsp` files by default from a JAR.	Changed `<packaging>` to `war` and added `tomcat-embed-jasper` with scope `provided`, so JSPs are compiled and served correctly under `src/main/webapp/WEB-INF/views`.
Listing books in JSP triggered `LazyInitializationException` because the `Book.author` relationship is fetched lazily and the JSP renders after the transaction has closed.	Avoided the issue entirely by introducing a `BookAuthorDTO` projection and a JPQL query that **INNER JOIN**s on `b.author`, returning the joined fields in a single statement. This also fulfilled the assignment's join requirement.
Need to surface integrity violations (duplicate ISBN, duplicate email) cleanly to the user instead of a 500 page.	Two-layer handling: the service throws `DataIntegrityViolationException` with a friendly message; the controller catches it and re-renders the form with a field-level error via `BindingResult.rejectValue(...)`. A `@ControllerAdvice` handler is the safety-net for anything the controller does not catch.

Local environment had no Maven installed, which would block the grader from building the project.	Added the Maven Wrapper (`./mvnw` , `./mvnw.cmd` , `.mvn/wrapper/`) so anyone with just a JDK can run `./mvnw spring-boot:run` without installing Maven first.

7. Build & Run Instructions

```
# Run unit tests (17 tests)
./mvnw test

# Start the application
./mvnw spring-boot:run

# Then open in a browser:
#   http://localhost:8080/             -> home
#   http://localhost:8080/authors      -> authors list
#   http://localhost:8080/books        -> books list (uses INNER JOIN)
#   http://localhost:8080/h2-console   -> H2 console (JDBC URL: jdbc:h2:mem:librarydb)
```

8. GitHub Repository

Source code is hosted publicly on GitHub:

https://github.com/Krritin/library-management

To clone and run locally:

```
git clone https://github.com/Krritin/library-management.git
cd library-management
./mvnw spring-boot:run
```